JOBSHEET 5

NAMA : M. Javier Thufail

NIM : 254107020019

KELAS : TI 1-G

NO. ABSEN : 18

PERCOBAAN 1 (Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer).

Code Class:

Faktorial18.java U | MainFaktorial18.java U | Pangkat18.java U | Sum18.java U

Jobsheet5 > Faktorial18.java > Faktorial18

```java
package Jobsheet5;

public class Faktorial18 {

    int faktorialBF(int n){
        int fakto = 1;
        for(int i = 1; i <= n; i++){
            fakto = fakto * i;
        }
        return fakto;
    }

    int faktorialDC(int n){
        if(n == 1){
            return 1;
        }else{
            int fakto = n * faktorialDC(n - 1);
            return fakto;
        }
    }
}
```

Code Main Class dan Outputnya:

Faktorial18.java U | MainFaktorial18.java U | Pangkat18.java U | MainPangkat18.java U | Sum18.java U

Jobsheet5 > MainFaktorial18.java > ...

```java
package Jobsheet5;

import java.util.Scanner;

public class MainFaktorial18 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print(s: "Masukkan nilai: ");
        int nilai = input.nextInt();

        Faktorial18 fk = new Faktorial18();
        System.out.println("Nilai faktorial " + nilai + " menggunakan BF: " + fk.faktorialBF(nilai));
        System.out.println("Nilai faktorial " + nilai + " menggunakan DC: " + fk.faktorialDC(nilai));
        input.close();
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData>  & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionM
e\User\workspaceStorage\536fd784924cf6d1cf999570c8a955a3\redhat.java\jdt_ws\PraktikumAlgoritmaStrukturData_b46f464f\bin' 'Job
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData>
```

Jawaban Pertanyaan!

1. - Code if untuk menghentikan rekursi apabila nilai sudah 1 nilai akan dikembalikan dan diberhentikan. Sedangkan,

   - Code else untuk melanjutkan perhitungan faktorial secara rekursif.

2. Perulangan pada method faktorial() **bisa** menggunakan selain for.

   Faktorial18.java U MainFaktorial18.java U

   Jobsheet5 > Faktorial18.java > Faktorial18 > fak

```java
package Jobsheet5;

public class Faktorial18 {

    int faktorialBF(int n){
        int fakto = 1;
        int i = 1;
        while (i <= n) {
            fakto = fakto * i;
            i++;
        }
        return fakto;
    }
```

   menggunakan while juga program tetap bisa dirun.

3. - fakto *= i; digunakan pada metode algoritma brute force perhitungan faktorial menggunakan perulangan. Sedangkan,

   - int fakto = n * faktorialDC(n-1); digunakan pada metode algoritma divide conquer, perhitungan faktorial menggunakan rekursi.

4. Method faktorialBF() menghitung faktorial dengan perulangan, Sedangkan faktorialDC() menghitung faktorial dengan rekursi yang memanggil fungsi sendiri sampai kondisi dasar tercapai.

PERCOBAAN 2 (Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer).

Code class:

   Pangkat18.java U MainPangkat18.java U Faktorial18.java U MainFaktorial18.java U Sum18.java U

   Jobsheet5 > Pangkat18.java > ...

```java
package Jobsheet5;

public class Pangkat18 {
    int nilai, pangkat;

    Pangkat18(int n, int p){
        nilai = n;
        pangkat = p;
    }

    int pangkatBF(int a, int n){
        int hasil = 1;
        for(int i = 0; i < n; i++){
            hasil = hasil * a;
        }
        return hasil;
    }

    int pangkatDC(int a, int n){
        if(n == 1){
            return a;
        }else{
            if(n % 2 == 1){
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
            }else{
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
            }
        }
    }
}
```

Code Main Class dan Outputnya:

```java
package Jobsheet5;

import java.util.Scanner;

public class MainPangkat18 {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print(s: "Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Pangkat18[] png = new Pangkat18[elemen];

        for(int i = 0; i < elemen; i++){
            System.out.print("Masukkan nilai basis elemen ke-" + (i+1) + ": ");
            int basis = input.nextInt();

            System.out.print("Masukkan nilai pangkat elemen ke-" + (i+1) + ": ");
            int pangkat = input.nextInt();

            png[i] = new Pangkat18(basis, pangkat);
        }

        System.out.println(x: "HASIL PANGKAT BRUTEFORCE:");
        for(Pangkat18 p : png){
            System.out.println(p.nilai + "^" + p.pangkat + " = " +
                    p.pangkatBF(p.nilai, p.pangkat));
        }

        System.out.println(x: "HASIL PANGKAT DIVIDE AND CONQUER:");
        for(Pangkat18 p : png){
            System.out.println(p.nilai + "^" + p.pangkat + " = " +
                    p.pangkatDC(p.nilai, p.pangkat));
        }
        input.close();
    }
}
```

```
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData>  & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\user\AppData\Roaming
ws\PraktikumAlgoritmaStrukturData_b46f464f\bin' 'Jobsheet5.MainPangkat18'
Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3 = 8
4^5 = 1024
6^7 = 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3 = 8
4^5 = 1024
6^7 = 279936
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData>
```

Jawaban Pertanyaan!

1. - pangkatBF() menghitung pangkat menggunakan perulangan dengan mengalikan nilai basis secara berulang sampai jumlah pangkat terpenuhi. Sedangkan,

   - pangkatDC() menghitung pangkat menggunakan rekursi dengan membagi perhitungan pangkat menjadi bagian yang lebih kecil lalu mengembalikannya kembali.

2. Ya, tahap combine sudah termasuk dalam kode tersebut, yaitu pada saat hasil pemanggilan rekursi dikalikan kembali.

   return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a); dan

   return (pangkatDC(a, n/2) * pangkatDC(a, n/2));

3. Menurut saya method tersebut tetap relavan. den method tetap bisa dibuat tanpa parameter dan tetap berjalan dengan benar.

```java
int pangkatBF(int a, int n){
    int hasil = 1;
    for(int i = 0; i < n; i++){
        hasil = hasil * a;
    }
    return hasil;
}
```

4. Pada method pangkatBD menggunakan metode brute force. sedangkan, pada method pangkat DC menggunakan metode divide conquer.

PERCOBAAN 3 (Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer).

Code Class:

```java
package Jobsheet5;

public class Sum18 {
    double keuntungan[];

    Sum18(int el){
        keuntungan = new double[el];
    }

    double totalBF(){
        double total = 0;
        for(int i = 0; i < keuntungan.length; i++){
            total = total + keuntungan[i];
        }
        return total;
    }

    double totalDC(double arr[], int l, int r){
        if(l == r){
            return arr[l];
        }

        int mid = (l + r) / 2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid + 1, r);
        return lsum + rsum;
    }
}
```

Code Main Class dan Outputnya:

```java
package Jobsheet5;

import java.util.Scanner;

public class MainSum18 {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Sum18 sm = new Sum18(elemen);

        for(int i = 0; i < elemen; i++){
            System.out.print("Masukkan keuntungan ke-" + (i+1) + ": ");
            sm.keuntungan[i] = input.nextDouble();
        }

        System.out.println("Total keuntungan menggunakan BruteForce: " + sm.totalBF());
        System.out.println("Total keuntungan menggunakan Divide and Conquer: " + sm.totalDC(sm.keuntungan, 0, elemen-1));
        input.close();
    }
}
```

```
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\user\
ws\PraktikumAlgoritmaStrukturData_b46f464f\bin' 'Jobsheet5.MainSum18'
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan BruteForce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
```

Jawaban Pertanyaan!

1. Variabel mid dibutuhkan untuk menentukan titik pembagian array menjadi dua bagian agar proses Divide and Conquer bisa dilakukan.
2. Kedua statement tersebut digunakan untuk menghitung total masing-masing bagian array secara rekursif, yang kemudian akan digabungkan (lsum + rsum) untuk mendapatkan total keseluruhan (dalam metode divide conquer).
3. Penjumlahan ini dilakukan untuk menggabungkan hasil total bagian kiri dan kanan agar mendapatkan total seluruh array (tahap divide conquer combine).
4. base case:

```
if(l == r){
    return arr[l];
}
```

5. Method totalDC() bekerja dengan cara membagi array menjadi dua bagian, kemudian menghitung total masing-masing bagian secara rekursif, lalu menggabungkan hasilnya dengan penjumlahan sampai diperoleh total seluruh elemen array.

Latihan Praktikum.

Code Main Class dan Outputnya:

Mahasiswa18.java U | MainMahasiswa18.java U | Sum18.java | MainSum18.java | Pangkat18.java | MainPang

Jobsheet5 > TugasJobsheet5 > MainMahasiswa18.java > MainMahasiswa18

```
package Jobsheet5.TugasJobsheet5;

public class MainMahasiswa18 {
    Run | Debug
    public static void main(String[] args) {

        Mahasiswa18 data[] = {
            new Mahasiswa18(nama: "Ahmad",nim: "220101001",tahunMasuk: 2022,nilaiUTS: 78,nilaiUAS: 82),
            new Mahasiswa18(nama: "Budi",nim: "220101002",tahunMasuk: 2022,nilaiUTS: 85,nilaiUAS: 88),
            new Mahasiswa18(nama: "Cindy",nim: "220101003",tahunMasuk: 2021,nilaiUTS: 90,nilaiUAS: 87),
            new Mahasiswa18(nama: "Dian",nim: "220101004",tahunMasuk: 2021,nilaiUTS: 76,nilaiUAS: 79),
            new Mahasiswa18(nama: "Eko",nim: "220101005",tahunMasuk: 2023,nilaiUTS: 92,nilaiUAS: 95),
            new Mahasiswa18(nama: "Fajar",nim: "220101006",tahunMasuk: 2020,nilaiUTS: 88,nilaiUAS: 85),
            new Mahasiswa18(nama: "Gina",nim: "220101007",tahunMasuk: 2023,nilaiUTS: 80,nilaiUAS: 83),
            new Mahasiswa18(nama: "Hadi",nim: "220101008",tahunMasuk: 2020,nilaiUTS: 82,nilaiUAS: 84)
        };

        int maxUTS = Mahasiswa18.maxUTS(data, l: 0, data.length-1);
        int minUTS = Mahasiswa18.minUTS(data, l: 0, data.length-1);
        double rataUAS = Mahasiswa18.rataUAS(data);

        System.out.println("Nilai UTS Tertinggi : " + maxUTS);
        System.out.println("Nilai UTS Terendah  : " + minUTS);
        System.out.println("Rata-rata Nilai UAS : " + rataUAS);
    }
}
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | GITLENS

```
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData>  & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMes
ws\PraktikumAlgoritmaStrukturData_b46f464f\bin' 'Jobsheet5.TugasJobsheet5.MainMahasiswa18'
Nilai UTS Tertinggi : 92
Nilai UTS Terendah  : 76
Rata-rata Nilai UAS : 85.375
PS C:\TI\ASD\PraktikumAlgoritmaStrukturData>
```

Code Class:

Mahasiswa18.java U | MainMahasiswa18.java U | Sum18.java | MainSum18.java | Pangkat18.java | MainPangkat18.java | Faktor

Jobsheet5 > TugasJobsheet5 > Mahasiswa18.java > Mahasiswa18

```java
package Jobsheet5.TugasJobsheet5;

public class Mahasiswa18 {

    String nama;
    String nim;
    int tahunMasuk;
    int nilaiUTS;
    int nilaiUAS;

    Mahasiswa18(String nama, String nim, int tahunMasuk, int nilaiUTS, int nilaiUAS){
        this.nama = nama;
        this.nim = nim;
        this.tahunMasuk = tahunMasuk;
        this.nilaiUTS = nilaiUTS;
        this.nilaiUAS = nilaiUAS;
    }

    // Divide and Conquer mencari UTS tertinggi
    static int maxUTS(Mahasiswa18 arr[], int l, int r){
        if(l == r){
            return arr[l].nilaiUTS;
        }

        int mid = (l + r) / 2;

        int kiri = maxUTS(arr, l, mid);
        int kanan = maxUTS(arr, mid + 1, r);

        return Math.max(kiri, kanan);
    }

    // Divide and Conquer mencari UTS terendah
    static int minUTS(Mahasiswa18 arr[], int l, int r){
        if(l == r){
            return arr[l].nilaiUTS;
        }

        int mid = (l + r) / 2;

        int kiri = minUTS(arr, l, mid);
        int kanan = minUTS(arr, mid + 1, r);

        return Math.min(kiri, kanan);
    }

    // Brute Force menghitung rata-rata UAS
    static double rataUAS(Mahasiswa18 arr[]){
        int total = 0;

        for(int i = 0; i < arr.length; i++){
            total += arr[i].nilaiUAS;
        }

        return (double) total / arr.length;
    }
}
```